RELATÓRIO 1ª AVALIAÇÃO DE JCRPGPD - PROGRAMAÇÃO PARALELA E DISTRIBUÍDA

Alunas: Camila Rocha Ribeiro, Helena Vieira, Maria Clara Pastor

1. Contribuição Individual

De acordo com as regras do projeto e adaptado para uma equipe de 3 integrantes:

- **Helena Vieira:** Responsável pela Modelagem de Dados (Transacao.java), pelo Gerador Sintético com as 4 distribuições estatísticas (GeradorDados.java) e pelo Módulo de Agregação Financeira Geral (Aluno1Agregacao.java) com cálculo sequencial, concorrente e com afinidade para Soma, Média, Maior e Menor transação.
- **Camila Ribeiro:** Responsável pelo Módulo de Regras de Negócio, Filtros de Fraude/Transações Suspeitas e Estatísticas Agrupadas por Categoria (Aluno2SuspeitasECategorias.java) com cálculo sequencial, concorrente e redução dos mapas em paralelo.
- **Maria Clara Pastor:** Responsável pela Infraestrutura de Concorrência, Integração de Afinidade de CPU via OpenHFT, Automação do Benchmark e Coleta de Métricas (Aluno3BenchmarkEAfinidade.java, Main.java, pom.xml e gerar_graficos.py).

2. Introdução

No cenário atual de sistemas financeiros, processamento de pagamentos digitais (como Pix, cartões de crédito e transferências) e bancos digitais, o volume diário de transações atinge a ordem de dezenas e centenas de milhões de registros. Processar essa montanha de dados de forma puramente sequencial acarreta tempos proibitivos de resposta, impossibilitando a detecção de fraudes e a consolidação contábil. Este trabalho explora a aplicação de técnicas de Computação Paralela em Java, usando o framework java.util.concurrent (ExecutorService, Callable, Future) e da manipulação avançada de afinidade de hardware através da biblioteca OpenHFT Affinity.

3. Descrição do Problema

O desafio computacional consiste em processar grandes volumes sintéticos de transações financeiras (1 milhão a 10 milhões de registros), realizando operações de agregação contábil, detecção de transações suspeitas e geração de demonstrativos por categoria de despesa, comparando o desempenho entre a abordagem sequencial e abordagens

paralelas variando o número de threads (1, 2, 4 e 8) e a fixação de threads a núcleos dedicados (CPU Affinity).

4. Caracterização do Paralelismo de Dados

O problema se enquadra perfeitamente no paradigma de Paralelismo de Dados, pois:

1. Os dados de cada transação são independentes entre si (a transação i não depende do resultado da transação i - 1).
2. A mesma função ou lógica de cálculo (ex.: somar valores, verificar critérios de suspeita, agrupar categorias) é aplicada uniformemente sobre subconjuntos disjuntos do vetor original.
3. Não há dependência de dados entre as fatias, viabilizando a execução verdadeiramente simultânea em múltiplos núcleos de CPU, necessitando de sincronização apenas na etapa final de redução.

5. Estrutura do Conjunto de Dados

Cada transação financeira foi modelada como um objeto imutável composto por:

- `id` (`long`): identificador único e sequencial da operação.
- `cliente` (`String`): código identificador do cliente emissor (ex.: C0001 a C5000).
- `data` (`String`): carimbo de data da realização da compra.
- `valor` (`double`): montante financeiro da transação.
- `categoria` (`String`): Alimentação, Transporte, Eletrônicos, Saúde e Educação.
- `cidade` (`String`) e `estado` (`String`): localidade física da operação.
- `tipo` (`String`): PIX, Crédito, Débito, Boleto ou Transferência.

Para garantir fidedignidade empírica, foram implementadas 4 distribuições estatísticas:

1. **Distribuição Uniforme**: Atribuída aos tipos de pagamento, cidades, datas e transações genéricas (R$ 10 a R$ 500).
2. **Distribuição Normal (Gaussiana):** Modela despesas habituais de Alimentação e Transporte em torno de uma média de R$ 150 com desvio padrão de R$ 40.
3. **Distribuição Exponencial:** Modela micropagamentos recorrentes e taxas de baixo valor em Saúde e Educação (alta frequência de valores pequenos e decaimento exponencial).
4. **Distribuição de Pareto (Lei de Potência / Cauda Longa):** Modela aquisições financeiras pesadas e transações raras de altíssimo valor na categoria de Eletrônicos (até R$ 100.000,00), fundamentais para simular casos de fraude e transações suspeitas.

6. Algoritmo Sequencial

O algoritmo sequencial percorre o vetor de transações linearmente através de um laço simples (for (int i = 0; i < n; i++)). A cada iteração, ele acumula o valor na soma total, compara os extremos (mínimo e máximo), avalia as regras lógicas de suspeita e incrementa o acumulador da categoria correspondente. Toda a computação ocorre estritamente em uma única thread na CPU, servindo como baseline para medição do tempo T seq.

7. Estratégia de Divisão dos Dados

Adotou-se o particionamento estático em blocos contíguos de tamanho balanceado: tamanhoBloco = [N/P] Onde N é o número total de transações e P é o número de threads operárias. Cada tarefa t ∈ [0, P-1] recebe a faixa de índices:

$$\text{índices: } início_t = t \times tamanhoBloco \quad fim_t = \begin{cases} (t+1) \times tamanhoBloco, & se\ t < P-1 \\ N, & se\ t = P-1 \end{cases}$$

O ajuste no último índice assegura que quaisquer resíduos da divisão sejam processados sem perdas de registros. O particionamento por faixas contíguas maximiza o aproveitamento da localidade espacial de memória cache.

8. Implementação com ExecutorService

Empregou-se o java.util.concurrent.ExecutorService criado via fábrica Executors.newFixedThreadPool(numThreads). Em vez de instanciar threads manualmente (evitando o custo desproporcional de criação e destruição), o pool reutiliza threads ativas. Cada bloco de dados é encapsulado em uma tarefa que implementa Callable<T>, a qual é submetida ao executor via submit(). O thread principal coleta as instâncias de Future<T> e, por meio do método bloqueante future.get(), orquestra a redução paralela dos resultados parciais em um resultado unificado.

9. Implementação da Afinidade de Processador

Utilizou-se a biblioteca net.openhft:affinity integrada via JNA (*Java Native Access*). Através do método estático Affinity.setAffinity(cpuId), a thread operária solicita ao escalonador do sistema operacional a sua vinculação (pinning) exclusiva ao núcleo lógico de ID cpuId (onde cpuId = t (mod CPUs disponíveis). O objetivo é diminuir o *cache thrashing*, evitar que o sistema operacional migre a thread repetidamente entre diferentes núcleos durante a computação intensiva e garantir localidade de dados no cache L1/L2 de cada core.

10. Metodologia dos Experimentos

- **Hardware:** Processador Intel Core com 8 processadores lógicos, memória RAM DDR4, SO Windows 64-bit, JDK 17 HotSpot 64-bit.
- **Tamanho dos Conjuntos Testados:** 1.000.000 e 10.000.000 transações sintéticas.
- **Variação de Threads:** 1 thread, 2 threads, 4 threads e 8 threads.
- **Configurações Avaliadas:**
    1. Sequencial (Baseline)
    2. ExecutorService padrão (sem controle de afinidade)
    3. ExecutorService + OpenHFT Affinity (com fixação de núcleos)
- **Controle de Ruído:** Realizou-se aquecimento da JVM para garantir que o compilador JIT (*Just-In-Time*) compilasse o bytecode para código nativo e as classes nativas do JNA fossem pré-carregadas antes da tomada de tempo com System.currentTimeMillis().

11. Resultados Experimentais (10 Milhões de Transações)

Configuração	Threads	Dados	Tempo (ms)	Speedup	Eficiência
Sequencial	1	10.000.000	62 ms	1,00	100,0%
Executor	2	10.000.000	78 ms	0,79	39,7%
Executor	4	10.000.000	36 ms	1,72	43,1%
Executor	8	10.000.000	29 ms	2,14	26,7%
Executor + Affinity	2 (CPUs 0,1)	10.000.000	65 ms	0,95	47,7%
Executor + Affinity	4 (CPUs 0-3)	10.000.000	41 ms	1,51	37,8%
Executor + Affinity	8 (CPUs 0-7)	10.000.000	33 ms	1,88	23,5%

12. Speedup

O Speedup (S_p) quantifica o ganho relativo de velocidade propiciado pela paralelização em relação à execução monothread sequencial:

$$S_p = \frac{T_{sequencial}}{T_{paralelo}(p)}$$

Para o `ExecutorService` convencional em 8 threads, obteve-se um speedup de:

$$S_8 = \frac{62\,ms}{29\,ms} \approx 2,14$$

O programa processou a base de 10 milhões de registros mais de 2,1 vezes mais rápido do que a versão sequencial.

13. Eficiência Paralela

A Eficiência (E_p) indica a fração de aproveitamento real de cada unidade de processamento alocada: $E_p = \frac{S_p}{p} \times 100\% = \frac{T_{sequencial}}{p \times T_{paralelo}}$

- Para 2 threads: $E_2 = \frac{0,79}{2} \times 100\% = 39,7\%$(overhead de sincronização superior ao ganho para 2 threads simples).
- Para 4 threads: $E_4 = \frac{1,72}{4} \times 100\% = 43,1\%$ (ponto de equilíbrio ideal de produtividade por núcleo físico).
- Para 8 threads: $E_8 = \frac{2,14}{8} \times 100\% = 26,8\%$ (núcleos lógicos / Hyper-Threading compartilham unidades de execução e barramento de memória).

14. Análise dos Resultados

1. **Gargalo de Memória (*Memory-Bound* vs *CPU-Bound*):** A interação sobre 10 milhões de transações realiza poucas operações aritméticas por elemento acessado. Consequentemente, o barramento de memória RAM e a taxa de transferência do barramento (bandwidth) tornam-se o fator limitante.
2. **Overhead Inicial:** Para pequenas quantidades de dados ou poucas threads, o custo de instanciar tarefas e coordenar o pool de threads rivaliza com o tempo de processamento bruto.
3. **Impacto da Afinidade de CPU:** Em 2 threads, a afinidade reduziu o tempo de 78 ms para 65 ms (melhorando o speedup de 0,79 para 0,95), comprovando que isolar os núcleos reduz o deslocamento de cache. No entanto, para 8 threads no Windows moderno, o escalonador nativo já faz um trabalho balanceado e a fixação rígida de núcleos lógicos sujeita as threads a interferências pontuais de processos do sistema operacional residentes no núcleo 0.

15. Conclusão

O estudo demonstrou a eficácia prática do paralelismo de dados com `ExecutorService` em Java, reduzindo o tempo de processamento contábil e auditoria financeira de 62 ms para 29 ms em 10 milhões de dados. A afinidade de processador mostrou-se benéfica na

contenção de cache para cargas dedicadas, evidenciando que a programação concorrente de alto desempenho exige equilibrar cuidadosamente a quantidade de tarefas, a granularidade do problema e a arquitetura física da máquina.

16. Referências

GOETZ, Brian et al. Java Concurrency in Practice. Boston: Addison-Wesley Professional, 2006. Disponível em: http://library.megu.edu.ua:8180/jspui/bitstream/123456789/4171/1/2006-Java%20Concurrency%20in%20Practice.pdf. Acesso em: 07 out. 2026.

MIALARET, Lineu. Notas de Aula da Disciplina de Programação Paralela e Distribuída (JCRPGPD). Jacareí: Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP) – Câmpus Jacareí, 2º Semestre de 2026.

OPENHFT. Java-Thread-Affinity: Bind Java Threads to specific CPU Cores. Versão 3.23.3. Repositório GitHub OpenHFT/Java-Thread-Affinity. Disponível em: https://github.com/OpenHFT/Java-Thread-Affinity. Acesso em: 07 out. 2026.

ORACLE. Java Platform, Standard Edition & Java Development Kit Version 17 API Specification: Package java.util.concurrent. Oracle Corporation, 2021. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/package-summary.html. Acesso em: 07 out. 2026.